

МИНОБРНАУКИ РОССИИ

Федеральное государственное автономное образовательное учреждение высшего
образования

«Российский государственный гуманитарный университет»

(ФГАОУ ВО «РГГУ»)

Институт информационных наук и технологий безопасности

Факультет информационных систем и безопасности

Кафедра информационных
технологий и систем

Батагов Андрей Андреевич

Отчет по практической работе №3

по дисциплине «Программная инженерия»

студента 3 курса по направлению подготовки:

09.03.03 «Прикладная информатика»,

профиль: «Прикладная информатика в гуманитарной сфере»

Проверила:

Охалкина Е.П.

Отметка _____

Дата _____

Подпись _____

Москва 2026

Оглавление

Введение. Выбор темы и её актуальность.....	3
Описание объекта автоматизации	3
Постановка задачи.....	4
Формирование и анализ требований к ПО	4
UML проектирование	5
Выбор среды разработки	8
Описание процесса разработки	9
Описание работы готового ПО.....	12
Заключение	18
Список источников.....	19

Введение. Выбор темы и её актуальность

Современная программная инженерия немыслима без практических навыков создания полноценных веб-приложений, объединяющих бэкенд, фронтенд, базы данных и инфраструктуру контейнеризации. Банковская сфера остаётся одной из наиболее требовательных к безопасности, надёжности и юзабилити, а следовательно, представляет идеальную предметную область для отработки технологий промышленной разработки.

Демонстрационное банковское приложение выбрано темой работы, поскольку оно охватывает полный цикл проектирования и реализации:

- проектирование реляционной модели данных,
- разработку REST API с безопасной аутентификацией,
- создание современного адаптивного клиентского интерфейса,
- контейнеризацию сервисов с оркестрацией через Docker Compose и обратное проксирование через Nginx.

Актуальность темы обусловлена широким применением использованных технологий, FastAPI, Next.js, TypeScript, PostgreSQL, Docker, в реальных проектах финансовой и других отраслей. Выполненная работа демонстрирует владение полным инструментальным стеком, умение формулировать требования, проектировать архитектуру и реализовывать готовое ПО.

Цель работы: разработать демонстрационное банковское приложение с базовыми операциями, регистрация, аутентификация, создание счетов и карт, переводы, история транзакций, обеспечив высокий уровень безопасности и удобства развёртывания.

Описание объекта автоматизации

Объектом автоматизации является упрощённая банковская информационная система для обслуживания частных клиентов. В реальности подобная система ответственна за хранение данных о клиентах, счетах, банковских картах и выполнение финансовых транзакций. В рамках учебного проекта реализована минимально необходимая функциональность, позволяющая продемонстрировать ключевые бизнес-процессы.

Основные сущности предметной области:

- **Пользователь (Client)** — физическое лицо, регистрирующееся в системе. Характеризуется именем пользователя и хэшированным паролем.
- **Счёт (Account)** — банковский счёт, принадлежащий пользователю. Имеет уникальный 20-значный номер, баланс и статус, активен/закрыт.
- **Карта (Card)** — банковская карта, связанная со счётом. Номер карты состоит из 16 цифр и проходит валидацию по алгоритму Луна.
- **Транзакция (Transaction)** — запись о перемещении средств между двумя счетами. Фиксирует сумму, время и тип операции.

Информационная система должна обеспечивать безопасный доступ пользователей, проведение переводов, просмотр истории и управление счетами. Взаимодействие с пользователем осуществляется через веб-интерфейс, вся бизнес-логика сосредоточена на бэкенд-сервере, а данные хранятся в реляционной СУБД.

Постановка задачи

Требуется создать демонстрационное банковское приложение, состоящее из трёх основных компонентов: серверной части, FastAPI, клиентской части, Next.js 14 и базы данных, PostgreSQL 16, разворачиваемых с помощью Docker Compose и Nginx.

Перед проектом ставятся следующие функциональные задачи:

1. Реализовать регистрацию нового пользователя с хэшированием пароля, bcrypt.
2. Обеспечить вход в систему с выдачей JWT-токена, python-jose.
3. При регистрации автоматически создавать счёт с 20-значным номером и банковскую карту с 16-значным номером, валидным по Луну.
4. Предоставить защищённые эндпоинты для перевода средств, с проверкой баланса и блокировкой запрещённых операций, просмотра истории транзакций и закрытия счёта.
5. На клиентской стороне реализовать страницы входа, регистрации, панель управления, dashboard, с разделами «Счета», «Переводы», «История»; адаптивный дизайн и мобильное меню.
6. Настроить автоматическую подстановку Bearer-токена в запросы через Axios-клиент и защитить маршруты, редирект на /login при отсутствии токена.
7. Сконфигурировать CORS middleware для безопасного взаимодействия.
8. Упаковать все компоненты в Docker-образы, описать оркестрацию в docker-compose.yml, где backend стартует после health-check PostgreSQL, а Nginx направляет запросы /api/* на FastAPI, остальные — на Next.js.

Нефункциональные требования включают безопасность хранения учётных данных, кросс-браузерную поддержку, отзывчивость интерфейса и лёгкость развёртывания.

Формирование и анализ требований к ПО

На основе постановки задачи были сформулированы функциональные и нефункциональные требования.

Функциональные требования

- **Регистрация:** система должна позволять новому пользователю создать учётную запись, задав уникальное имя и пароль. Пароль хранится в виде bcrypt-хэша.
- **Аутентификация:** после ввода корректных учётных данных пользователь получает JWT-токен доступа.

- **Автоматическое создание счёта и карты:** при успешной регистрации генерируются счёт, 20 цифр и карта, 16 цифр, привязанные к пользователю.
- **Перевод средств:** авторизованный пользователь может перевести деньги с одного своего активного счёта на другой счёт, свой или чужой, по номеру. Проверяется достаточность средств и статус «активен» обоих счетов.
- **История транзакций:** пользователь может получить список всех операций по конкретному счёту.
- **Закрытие счёта:** возможность закрыть счёт, установить флаг `is_active = false`. После закрытия операции по нему блокируются.
- **Защита маршрутов:** клиентский интерфейс перенаправляет неавторизованных посетителей на страницу входа.
- **CORS:** бэкенд настроен на принятие запросов с доверенных источников фронтенд.

Нефункциональные требования

- **Безопасность:** пароли хэшируются bcrypt; JWT подписывается надёжным алгоритмом; номера карт генерируются по стандарту Луна, что предотвращает ввод случайных некорректных номеров.
- **Масштабируемость и развёртывание:** все компоненты контейнеризованы, запускаются единой командой `docker compose up`.
- **Отказоустойчивость БД:** backend ожидает health-check PostgreSQL перед стартом.
- **Интерфейс:** адаптивный дизайн, поддержка мобильных и десктопных устройств, с использованием Tailwind CSS.
- **Документирование API:** автоматическая генерация OpenAPI-спецификации силами FastAPI.

UML проектирование

Для визуализации архитектуры и поведения системы построены диаграммы UML.

Диаграмма прецедентов

Актёр: Пользователь, Клиент

Прецеденты:

- Регистрация
- Вход в систему
- Просмотр списка счетов
- Перевод средств
- Просмотр истории транзакций

- Заккрытие счёта

Отношения: все прецеденты, кроме Регистрации и Входа, требуют предварительной аутентификации связь включать в себя, с прецедентом «Вход в систему».

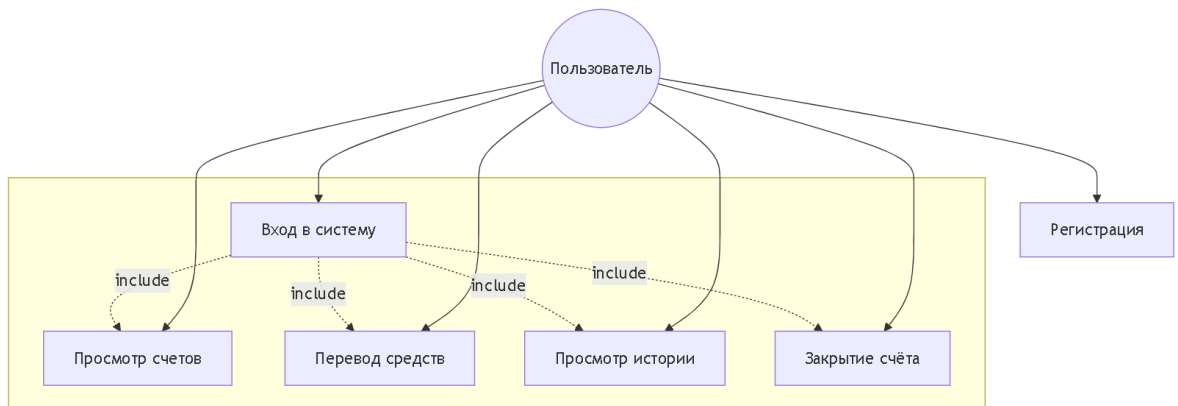


Рисунок 1.1. Диаграмма прецедентов.

Диаграмма последовательности сценарий «Перевод средств»

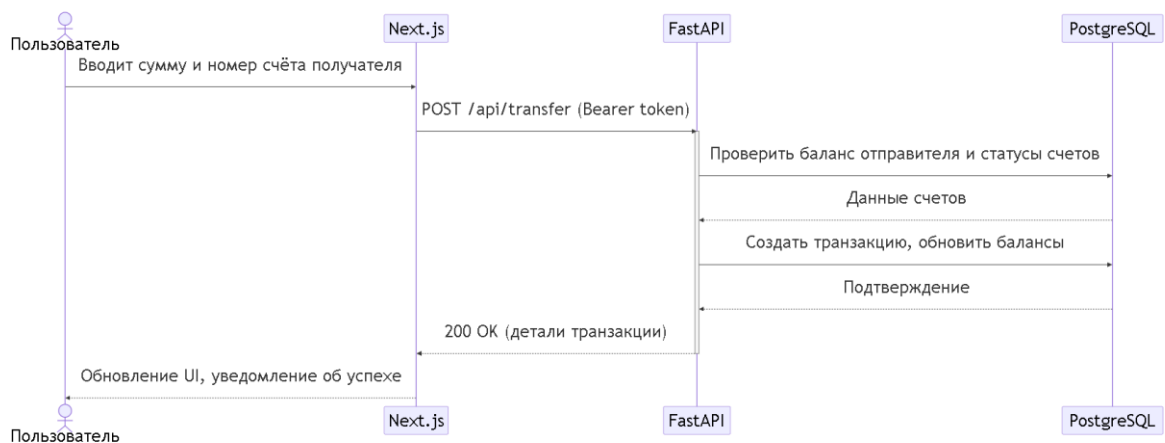


Рисунок 1.2. Диаграмма последовательности

Диаграмма классов

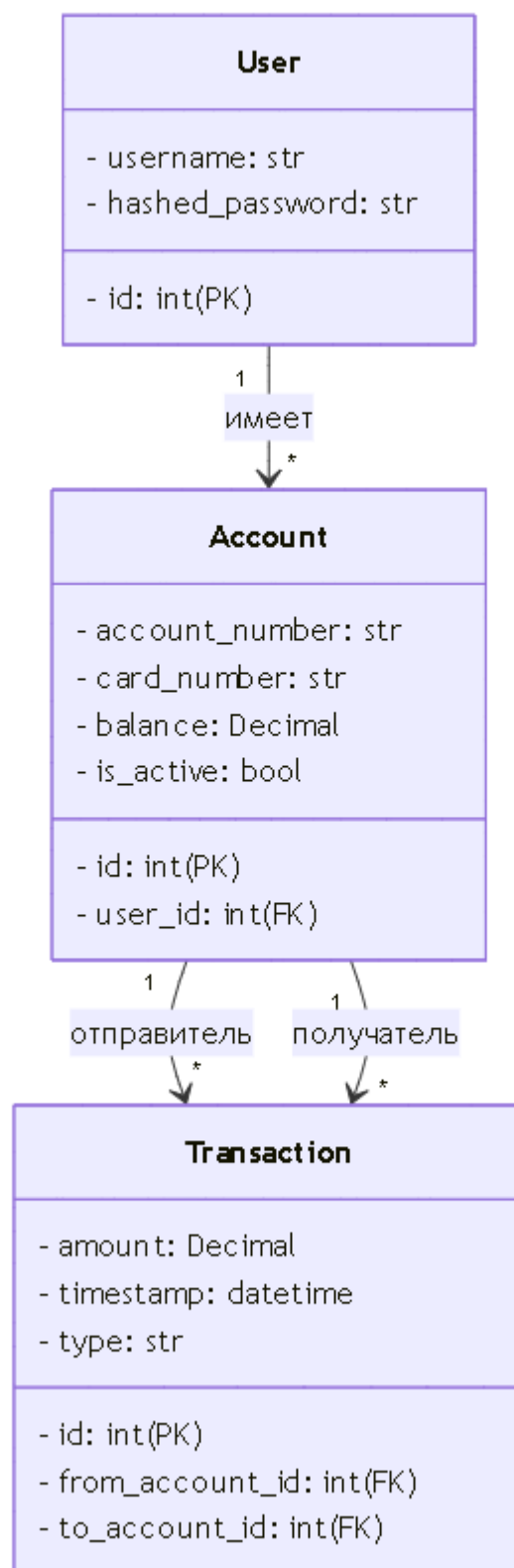


Рисунок 1.3. Диаграмма классов.

Диаграмма состояний, объект «Счёт»

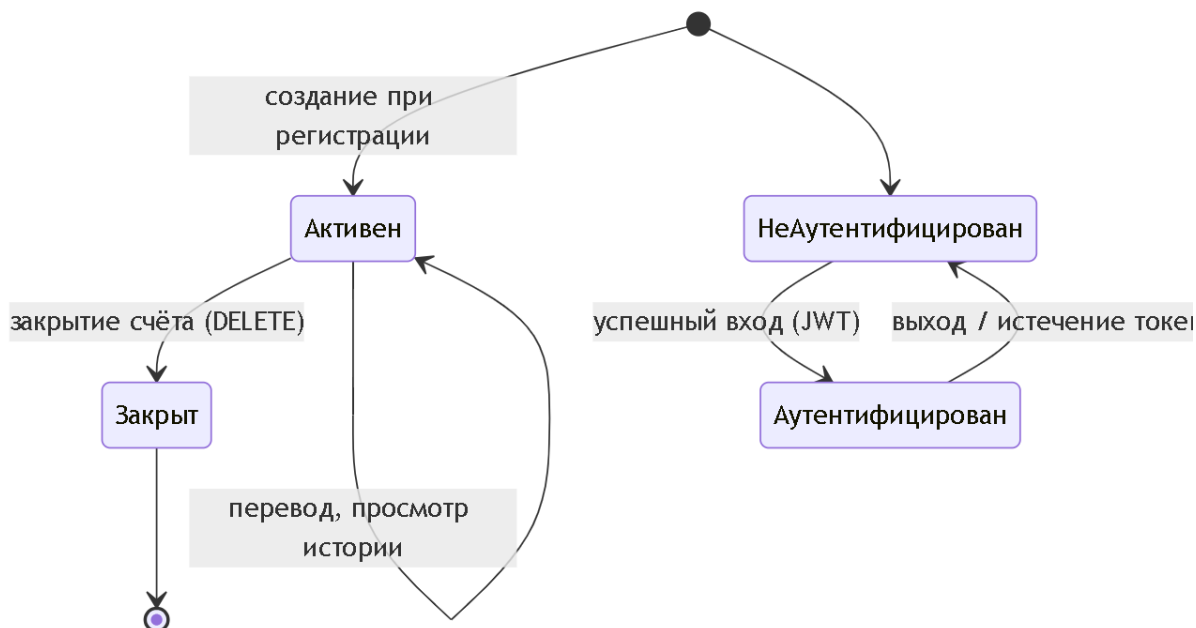


Рисунок 1.4. Диаграмма состояний.

Для пользовательской сессии можно выделить два состояния: **Не аутентифицирован** и **Аутентифицирован**, переход по успешному входу/выходу или истечению токена.

Выбор среды разработки

Серверная часть:

- **FastAPI Python 3.11+** - современный асинхронный веб-фреймворк с автоматической генерацией документации, встроенной поддержкой CORS, удобной интеграцией с Pydantic и SQLAlchemy.
- **python-jose** - библиотека для работы с JWT.
- **bcrypt** - надёжная реализация адаптивного хэширования паролей.
- **SQLAlchemy** - ORM для взаимодействия с PostgreSQL, позволяющая описывать модели декларативно.
- **PostgreSQL 16** - мощная реляционная СУБД с поддержкой транзакционной целостности.

Клиентская часть:

- **Next.js 14** - фреймворк на React с TypeScript, обеспечивающий маршрутизацию, серверный рендеринг, при необходимости, и удобную структуру приложения.
- **Tailwind CSS** - утилитарный CSS-фреймворк для быстрого создания адаптивных интерфейсов.

- **Axios** - HTTP-клиент с возможностью задания перехватчиков для автоматической вставки токена авторизации.

Инфраструктура:

- **Docker** и **Docker Compose** - контейнеризация всех компонентов, обеспечение единообразного окружения.
- **Nginx** - обратный прокси-сервер, маршрутизирующий запросы /api/* на FastAPI и всё остальное на Next.js.

Выбор данного стека обоснован его распространённостью в индустрии, высокой производительностью, простотой развёртывания и наличием обширной документации.

Описание процесса разработки

Разработка велась итеративно — сначала полностью поднимался один слой, тестировался изолированно, затем интегрировался со следующим. Это позволило локализовать ошибки на раннем этапе и не тащить проблемы через весь стек.

3.1. Проектирование моделей данных

На первом этапе были спроектированы четыре таблицы: users, accounts, cards и transactions. Для описания схемы использовался декларативный стиль SQLAlchemy 2.0 с аннотациями типов через Mapped и mapped_column, что обеспечивает строгую типизацию на уровне Python:

```
python
class UserTable(Base):
    __tablename__ = "users"
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String, nullable=False)
    phone: Mapped[str] = mapped_column(String, unique=True, nullable=False)
    password_hash: Mapped[str] = mapped_column(String, nullable=False)
    accounts: Mapped[list["AccountTable"]] = relationship(
        back_populates="user", cascade="all, delete-orphan"
    )
```

Связь между счётом и транзакцией потребовала особого внимания: один счёт может выступать и отправителем, и получателем. Это решено через два явных внешних ключа с указанием foreign_keys в каждом relationship:

```
python
sent_transactions: Mapped[list["TransactionTable"]] = relationship(
    foreign_keys="TransactionTable.sender_account_id",
    back_populates="sender_account"
)
received_transactions: Mapped[list["TransactionTable"]] = relationship(
    foreign_keys="TransactionTable.receiver_account_id",
    back_populates="receiver_account"
)
```

Создание таблиц происходит автоматически при старте приложения через `Base.metadata.create_all` внутри `lifespan`-контекста FastAPI — отдельные миграции Alembic не требуются для данного проекта.

3.2. Реализация бэкенда

Бэкенд построен на FastAPI с полностью асинхронным стеком: `asyncpg` в качестве драйвера PostgreSQL и `AsyncSession` из `SQLAlchemy`. Все запросы к БД выполняются без блокировки `event loop`.

Сессия создаётся через `async_sessionmaker` и передаётся в каждый эндпоинт через механизм зависимостей FastAPI (`Depends`):

```
python
AsyncSessionLocal = async_sessionmaker(
    bind=engine, class_=AsyncSession, expire_on_commit=False
)
```

```
async def get_session() -> AsyncSession:
    async with AsyncSessionLocal() as session:
        yield session
```

Для аутентификации реализована JWT-схема на основе библиотеки `python-jose`. При регистрации пароль хэшируется через `bcrypt` с 12 раундами, при входе — верифицируется через `passlib`:

```
python
def create_access_token(data: dict) -> str:
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=1440) # 24 часа
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm="HS256")
```

Защита эндпоинтов реализована через зависимость `get_current_user`, которая извлекает токен из заголовка `Authorization: Bearer <token>`, декодирует его и загружает пользователя из БД:

```
python
async def get_current_user(
    credentials: HTTPAuthorizationCredentials = Depends(security),
    session: AsyncSession = Depends(get_session),
) -> UserTable:
    payload = decode_access_token(credentials.credentials)
    user_id = payload.get("sub")
    result = await session.execute(
        select(UserTable).where(UserTable.id == int(user_id))
    )
    return result.scalar_one_or_none()
```

Эндпоинт перевода средств `POST /api/transfer` реализует атомарную логику: сначала проверяется владение счётом-отправителем и флаг `can_withdraw`, затем — достаточность баланса, после чего в рамках одной транзакции БД обновляются оба баланса и создаётся запись в `transactions`:

```
python
```

```

sender.balance -= data.amount
receiver.balance += data.amount
transaction = TransactionTable(
  sender_account_id=sender.id,
  receiver_account_id=receiver.id,
  amount=data.amount,
  description=data.description,
)
session.add(transaction)
await session.commit()

```

Номера счетов генерируются как 20-значные цифровые строки, номера карт — 16-значные с проверкой алгоритмом Луна, гарантирующим математическую корректность номера.

3.3. Разработка фронтенда

Фронтенд реализован на Next.js 14 с использованием App Router, TypeScript и Tailwind CSS. Аутентификация основана на JWT-токене, который сохраняется в cookie через библиотеку js-cookie со сроком жизни 24 часа.

Для взаимодействия с API создан централизованный экземпляр Axios с двумя перехватчиками. Первый автоматически добавляет заголовок авторизации к каждому запросу:

typescript

```

api.interceptors.request.use((config) => {
  const token = Cookies.get("token");
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

```

Второй перехватчик обрабатывает истёкшие или невалидные токены — при получении ответа 401 он удаляет cookie и перенаправляет пользователя на страницу входа:

typescript

```

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      Cookies.remove("token");
      window.location.href = "/login";
    }
    return Promise.reject(err);
  }
);

```

Защита роутов реализована в dashboard/layout.tsx: при монтировании компонент проверяет наличие токена и запрашивает профиль пользователя. Если токен отсутствует или запрос

вернул ошибку — происходит редирект на /login. Боковая навигация на мобильных устройствах сворачивается в бургер-меню через управление состоянием `useState` и оверлей поверх контента.

3.4. Контейнеризация

Сервисы упакованы в четыре Docker-контейнера, управляемых через `docker-compose.yml`. Образ фронтенда строится в три стадии (`multi-stage build`): установка зависимостей, сборка Next.js в `standalone`-режиме и минимальный `production`-образ, содержащий только скомпилированное приложение без исходников и `node_modules`. Это снижает итоговый размер образа примерно в 5–7 раз.

Бэкенд стартует только после того, как PostgreSQL пройдет `health-check` (`pg_isready`), что исключает ошибки подключения при инициализации. Nginx выступает единой точкой входа: запросы к `/api/` проксируются на `backend:8000`, всё остальное — на `frontend:3000`. Такая конфигурация устраняет CORS-проблемы, так как браузер видит только один `origin`.

3.5. Тестирование

Проведено ручное функциональное тестирование всех сценариев через интерфейс приложения и Swagger UI (`/docs`). Проверены граничные случаи: попытка перевода при недостатке средств возвращает `400 Insufficient funds`; перевод на несуществующий счёт — `404 Receiver account not found`; попытка снятия со счёта с флагом `can_withdraw=False` — `400 Withdrawals not allowed`. Проверена корректность работы CORS-middleware и цепочки редиректов на фронтенде при истёкшем токене.

Исходный код и инструкции по запуску размещены в репозитории с чёткой структурой.

Описание работы готового ПО

Предоставляю скрины интерфейса, которые могут быть использованы как инструкция для пользователей.

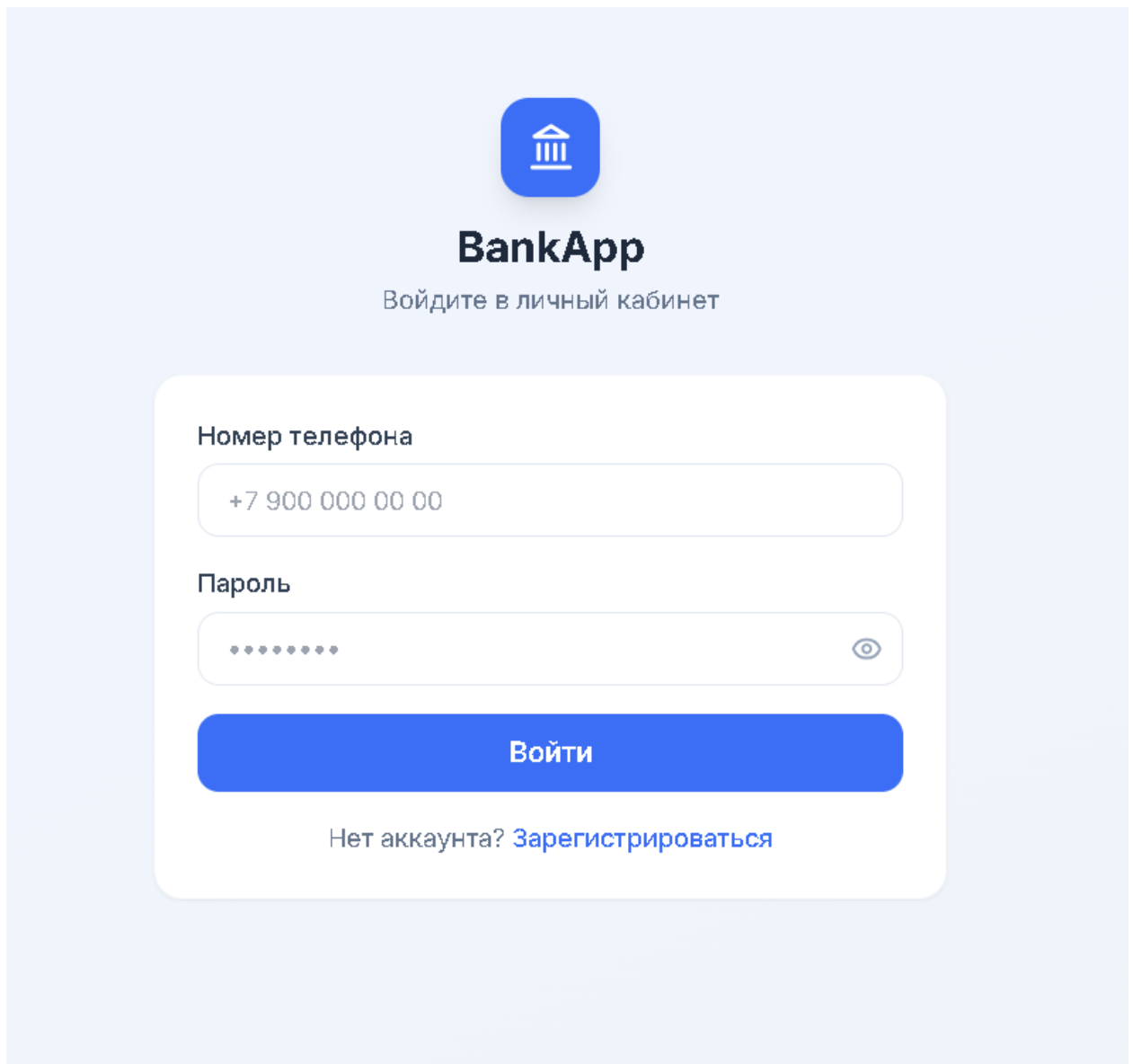


Рисунок 2.1. Окно авторизации

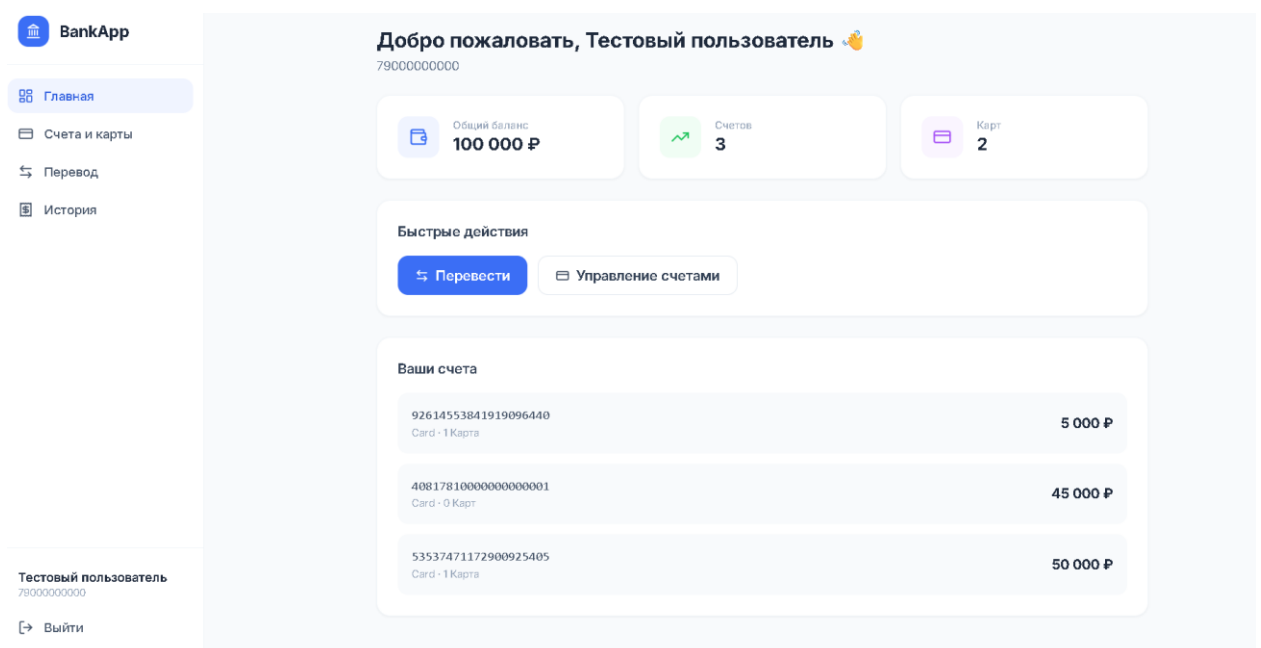


Рисунок 2.2. Начальный dashboard

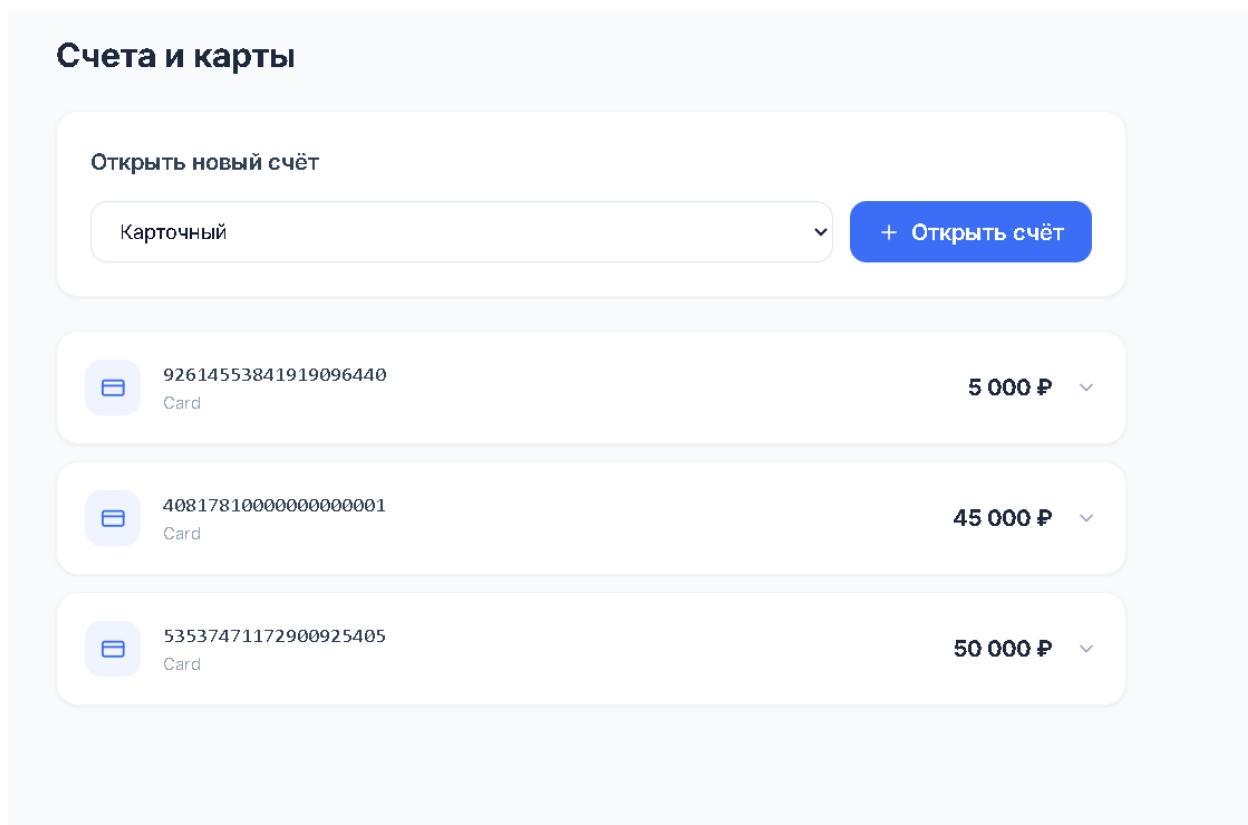


Рисунок 2.3. Управление картами

Перевод средств

Счёт отправителя

92614553841919096440 · 5 000 ₽



Доступно: 5 000 ₽



Счёт получателя

20-значный номер счёта

Сумма (₽)

0

Комментарий (необязательно)

Назначение платежа

⇄ Перевести

Рисунок 2.4. Окно перевода средств

История операций

Счёт

4081781000000000000001

Исходящий перевод

04 мая, 08:24 · #2

-50 000 ₽

Исходящий перевод

03 мая, 14:33 · #1

-5 000 ₽

Рисунок 2.5. История переводов по счёту.

auth		^
POST	/api/register Register	▼
POST	/api/login Login	▼
user		^
GET	/api/profile Profile	🔒 ▼
accounts		^
GET	/api/accounts Get Accounts	🔒 ▼
POST	/api/accounts Create Account	🔒 ▼
DELETE	/api/accounts/{account_number} Delete Account	🔒 ▼
cards		^
POST	/api/cards Add Card	🔒 ▼
transactions		^
POST	/api/transfer Transfer Money	🔒 ▼
GET	/api/transactions Get Transactions	🔒 ▼
system		^
GET	/api/health Health	▼

Рисунок 2.6. API эндпойнты.

Перевод средств



Перевод выполнен

Транзакция #3 · Остаток: 40 000 ₽

Счёт отправителя

53537471172900925405 · 40 000 ₽

Доступно: 40 000 ₽



Счёт получателя

92614553841919096440

Сумма (₽)

0

Комментарий {необязательно}

Назначение платежа

⇄ Перевести

Рисунок 2.7. Результат выполнения транзакции.

После запуска `docker compose up` все сервисы стартуют в следующем порядке:

1. Запускается PostgreSQL 16, проходит health-check.
2. За FastAPI поднимается бэкенд (порт 8000), который автоматически создаёт таблицы, если они отсутствуют.
3. Поднимается фронтенд (порт 3000).
4. Nginx (порт 80) начинает принимать запросы, проксируя их к соответствующим сервисам.

Пользователь открывает в браузере `http://localhost`. Его встречает страница входа. Если учётной записи нет, он переходит на `/register`, вводит желаемые логин и пароль. После успешной регистрации система автоматически создаёт первый банковский счёт, номер из 20 цифр и карту 16 цифр, валидных по Луна. Пользователь перенаправляется на страницу входа, где вводит свои данные.

При успешном входе бэкенд возвращает JWT-токен, который сохраняется в памяти клиента и автоматически подставляется во все последующие запросы через Axios-интерсептор. Пользователь попадает на `/dashboard`, где ему доступны через sidebar следующие разделы:

- **Счета** - отображает список счетов с номерами, балансами и номерами карт. Есть кнопка «Закрыть счёт», которая вызывает `DELETE /api/accounts/{number}` и делает счёт неактивным.
- **Переводы** - форма, в которой нужно выбрать свой счёт, ввести номер счёта получателя и сумму. При отправке выполняется POST-запрос; если операция прошла успешно, выводится сообщение, балансы обновляются. При ошибке, например, недостаточно средств, показывается предупреждение.
- **История транзакций** - позволяет выбрать счёт и просмотреть список всех входящих и исходящих переводов с временными метками и суммами.

Все страницы адаптивны: на мобильных устройствах боковое меню скрывается и вызывается по нажатию на иконку.

Бэкенд логирует запросы и ошибки, использует асинхронные сессии SQLAlchemy. Все пароли хранятся только в виде bcrypt-хэшей. Токены подписываются секретным ключом, задаваемым через переменную окружения.

Таким образом, готовое ПО полностью выполняет поставленные задачи и демонстрирует работу полноценного современного веб-приложения с безопасной аутентификацией.

Заключение

В ходе выполнения проекта было спроектировано и реализовано демонстрационное банковское приложение, охватывающее полный цикл веб-разработки: от анализа требований и моделирования данных до контейнеризации и деплоя. Использование FastAPI позволило быстро создать документированное и безопасное REST API, Next.js с TypeScript и Tailwind обеспечили современный адаптивный интерфейс, а Docker Compose с Nginx упростили запуск и масштабирование.

Решены ключевые задачи: регистрация и аутентификация с JWT, автоматическая генерация номеров счетов и карт с проверкой по алгоритму Луна, переводы средств с контролем баланса и статуса, история транзакций, закрытие счёта, защита клиентских маршрутов.

Полученный продукт демонстрирует навыки владения архитектурными шаблонами, UML-проектированием, выбором актуального технологического стека и умением интегрировать различные компоненты в единую систему. В дальнейшем проект может быть расширен

добавлением ролей администратор, поддержкой нескольких валют, полноценными юнит- и интеграционными тестами, мониторингом и CI/CD.

Выполненная работа подтверждает потенциал современных фреймворков и инструментов для быстрой разработки надёжных финансовых приложений.

Список источников

1. FastAPI Documentation. - Режим доступа: <https://fastapi.tiangolo.com/> (дата обращения: май 2026).
2. Next.js Documentation. - Режим доступа: <https://nextjs.org/docs> (дата обращения: май 2026).
3. TypeScript Handbook. - Режим доступа: <https://www.typescriptlang.org/docs/> (дата обращения: май 2026).
4. Tailwind CSS Documentation. - Режим доступа: <https://tailwindcss.com/docs> (дата обращения: май 2026).
5. SQLAlchemy 2.0 Documentation. - Режим доступа: <https://docs.sqlalchemy.org/en/20/> (дата обращения: май 2026).
6. PostgreSQL 16 Documentation. - Режим доступа: <https://www.postgresql.org/docs/16/> (дата обращения: май 2026).
7. Docker Compose Documentation. - Режим доступа: <https://docs.docker.com/compose/> (дата обращения: май 2026).
8. Nginx Reverse Proxy Guide. - Режим доступа: <https://docs.nginx.com/nginx/admin-guide/web-server/reverse-proxy/> (дата обращения: май 2026).
9. JWT Introduction. - Режим доступа: <https://jwt.io/introduction/> (дата обращения: май 2026).
10. python-jose documentation. - Режим доступа: <https://python-jose.readthedocs.io/> (дата обращения: май 2026).
11. bcrypt library documentation. - Режим доступа: <https://pypi.org/project/bcrypt/> (дата обращения: май 2026).